



Universidad
Indoamérica

UNITÉ 4 · FONDEMENTS DES SYSTÈMES D'EXPLOITATION

Processus et mémoire

Universidad Tecnológica Indoamérica

Laboratoire TD3

Le processus: un programme en cours d'exécution et son contexte

- Contexte: mémoire, registres, état des descripteurs...
- L'OS exécute de nombreux processus à la fois (unité 1, A1.4)
- Chacun a son propre **espace d'adressage privé**: isolation, sécurité et l'abstraction de la mémoire virtuelle
- Il cache le noyau et les autres processus

Adressage virtuel

- Une adresse de n bits permet 2^n adresses
- Espace d'adressage = les adresses + l'état de chaque case
- Chaque processus ne voit **que le sien**

Sur x86-64 aujourd'hui

Pointeurs de 64 bits, mais seuls **48** bits sont utilisés: 256 Tio d'espace virtuel, la moitié basse pour le processus et la moitié haute pour le noyau. Avec la pagination à 5 niveaux, 57 bits (128 Pio).

Les zones d'un processus: pile, tas, données, code

Sortie réelle

```
$ ./zones
```

zone		address	perm	region
code	(main)	0x5f935f9d1350	r-xp	~/fso/tds/bin/td3/zones
data	(initialized)	0x5f935f9d5010	rw-p	~/fso/tds/bin/td3/zones
bss	(global_array)	0x5f935f9d5040	rw-p	~/fso/tds/bin/td3/zones
heap	(malloc 10 B)	0x5f938dc96010	rw-p	[heap]
heap	(end: sbrk(0))	0x5f938dcb6fff	rw-p	[heap]
mmap	(malloc 64 MiB)	0x71b340fff010	rw-p	(anonymous)
libc	(printf)	0x71b345064080	r-xp	/usr/lib/x86_64-linux-gnu/libc.so.6
stack	(local)	0x7fff6c1e68b4	rw-p	[stack]

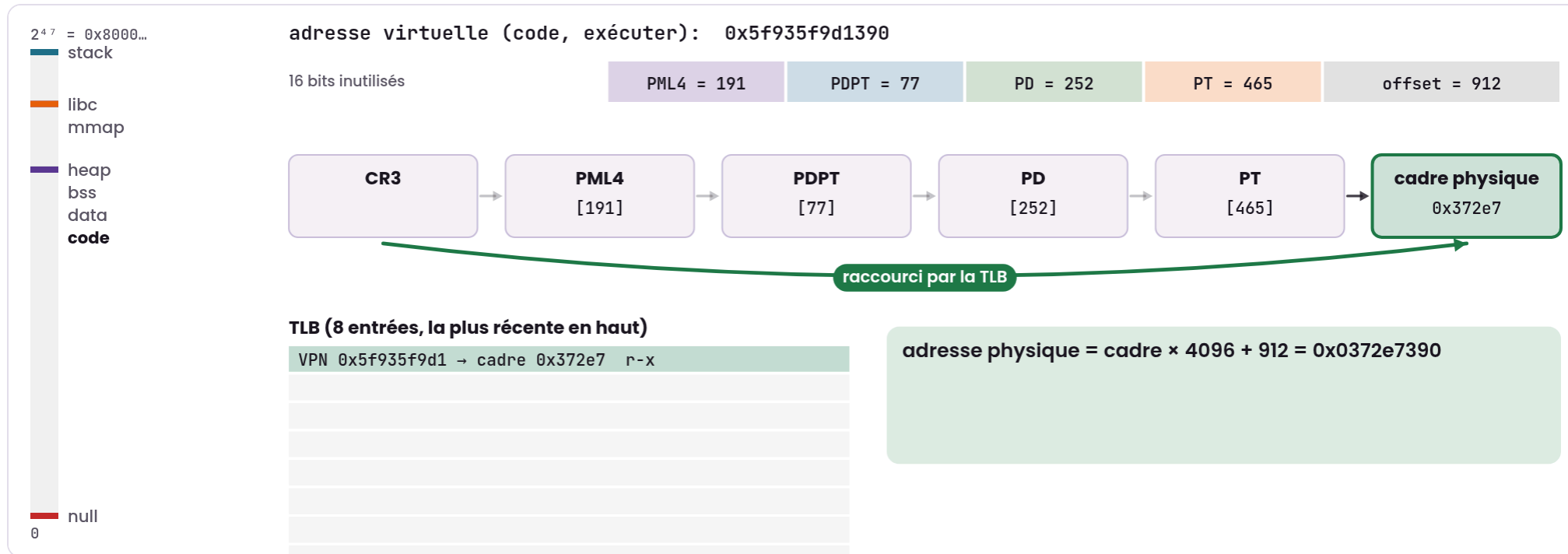
Pile : variables locales · **tas** : petits `malloc` · **mmap** : grands `malloc` et bibliothèques · **data/bss** : globales et `static` · **code** (text) : les instructions, en lecture seule et exécutables.

Du virtuel au physique: la pagination

- C'est le processeur qui traduit: la **MMU**, avec le cache **TLB**
- La mémoire est gérée en **pages** : 4 Kio sur x86-64 (et grandes pages de 2 Mio ou 1 Gio); sur ARM, aussi 16 ou 64 Kio
- Chaque page virtuelle pointe vers un **cadre** physique, par une table (sur x86-64, 4 niveaux)
- L'adresse est découpée en champs: indices de table et décalage

Correction de l'original: il disait « des blocs de 2 Ko (4 Ko ou 2 Mo sur x86) ». Sur x86, la page de base fait 4 Kio; 2 Ko n'est pas une taille de page x86.

Traduire des adresses réelles



Allocation paresseuse: la page existe quand on la touche

Sortie réelle

```
$ ./lazy 1024
start                                VmRSS =    1756 KiB    page faults so far = 0
malloc(1024 MiB) returned 0x7fadb67ff010
after malloc                          VmRSS =    1952 KiB    page faults so far = 8
after touching half of the pages     VmRSS =   526300 KiB    page faults so far = 131081
pages touched: 131072 (4096 bytes each) page faults: 131073    1968 ns per fault
after free                            VmRSS =    2012 KiB    page faults so far = 131081
```

- Bien plus efficace que de tout allouer d'avance
- Et s'il ne reste plus de cadres libres? Une page peu utilisée part sur le disque (swap) ou est compressée en RAM (zswap, zram): le remplacement de pages →

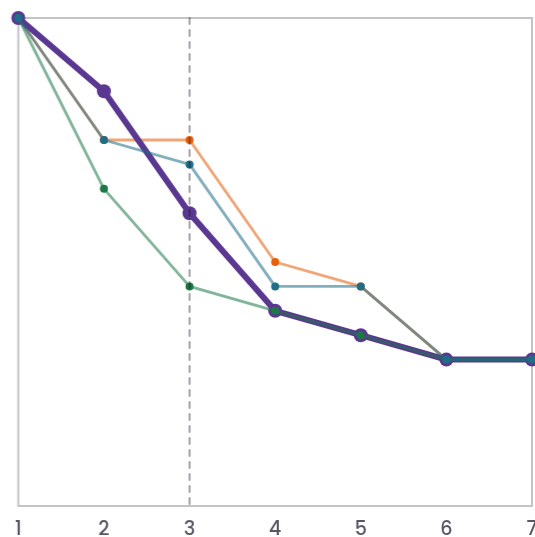
Plus de cadres libres: quelle page s'en va?

LRU · 3 cadres

réf.	7	0	1	2	0	3	0	4	2	3	0	3	2	1	2	0	1	7	0	1
cadre 0	7	7	7	2	2	2	2	4	4	4	0	0	0	1	1	1	1	1	1	1
cadre 1		0	0	0	0	0	0	0	0	3	3	3	3	3	3	0	0	0	0	0
cadre 2			1	1	1	3	3	3	2	2	2	2	2	2	2	2	2	7	7	7
	F	F	F	F		F		F	F	F	F			F		F		F		
				-7		-1		-2	-3	-0	-4			-0		-3		-2		

F = défaut · -x = victime

défauts selon les cadres



FIFO: 15

LRU: 12

Optimal (OPT): 9

L'interface mémoire: `brk`, `malloc`, `free`

C

```
int brk(void *end);           /* end of the heap */
void *sbrk(intptr_t delta);
void *malloc(size_t size);    /* NULL on failure */
void *calloc(size_t n, size_t size);
void *realloc(void *p, size_t size);
void free(void *p);          /* free(NULL): no-op */
```

N'utilisez pas `brk` / `sbrk` directement: utilisez `malloc`.

- glibc: petits blocs sur le tas (`brk`); à partir de 128 Kio, un `mmap` à part (visible dans `zones`)
- Linux est optimiste (*overcommit*): `malloc` ne renvoie presque jamais NULL; si la mémoire manque, l'**OOM killer** termine des processus
- Double `free` ou `free` de quelque chose de non alloué: comportement indéfini
- Débogage: GDB, Valgrind et aujourd'hui `-fsanitize=address` (ASan)

Le processus pour l'OS: le PCB

Statique

PID, PPID, propriétaire, droits d'accès

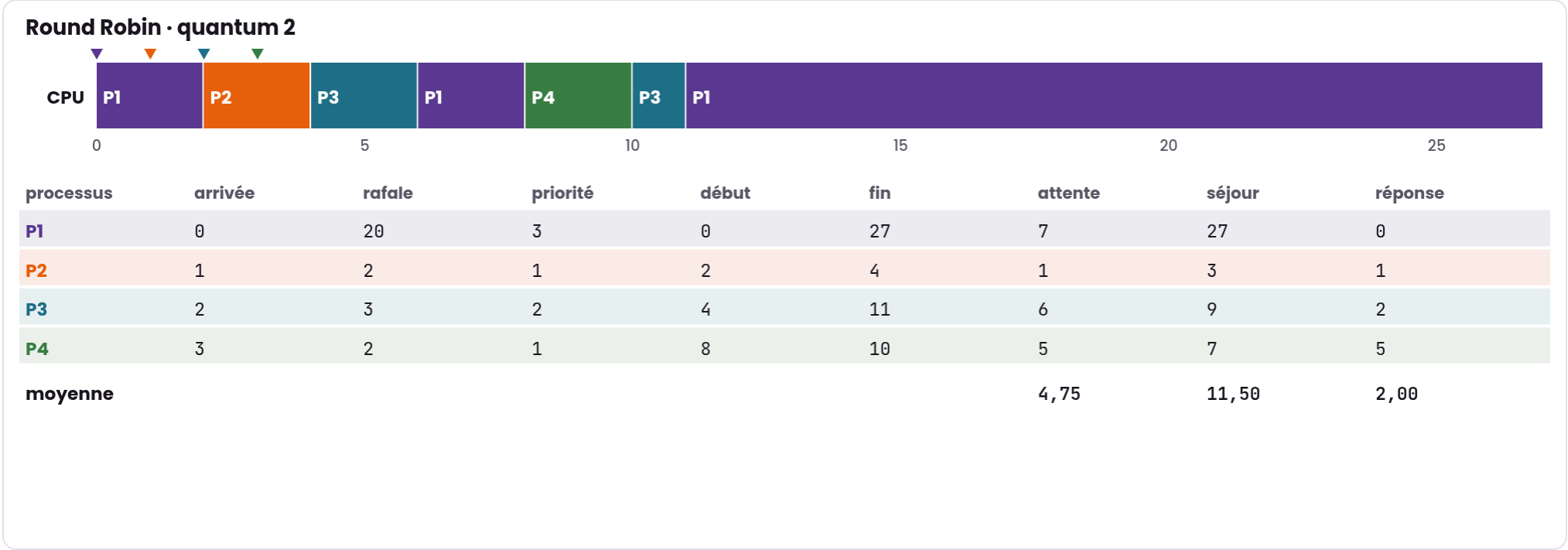
Dynamique

État, registres et compteur ordinal, priorité, temps processeur (utilisateur/système), descripteurs ouverts, limites (`getrlimit`), signaux, table des pages, threads

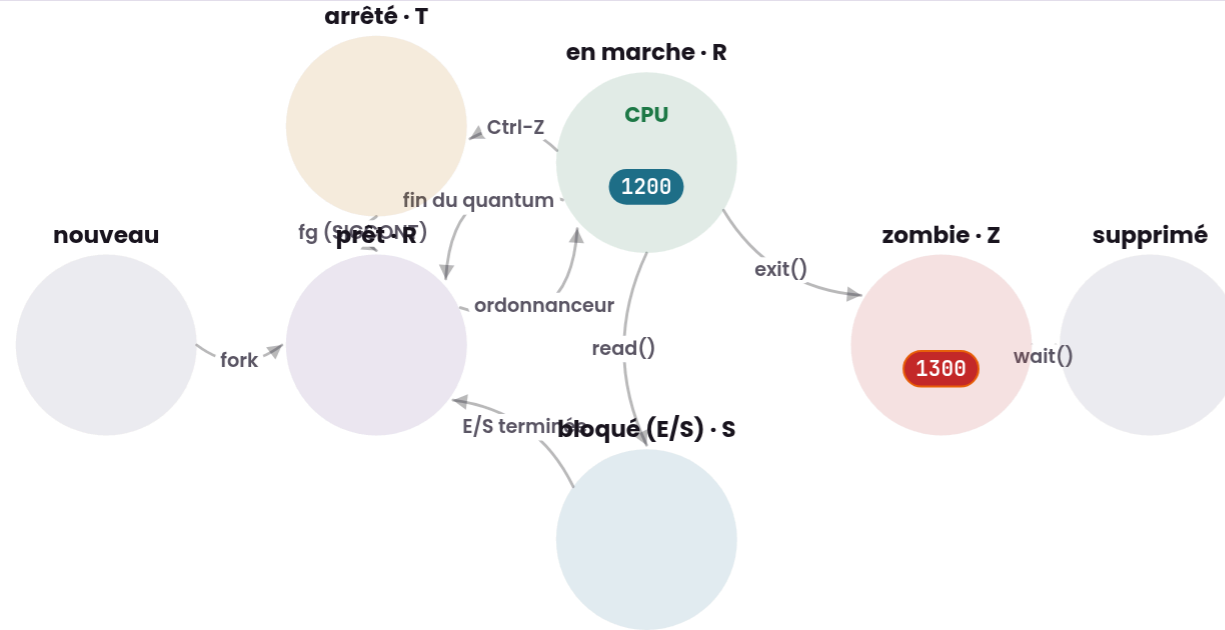
Sous Linux

Le PCB est `struct task_struct` (plusieurs Kio). On l'inspecte dans `/proc/PID/` : `status`, `maps`, `fd/`, `limits` ... C'est ainsi que fonctionnent `ps` et `top`.

Changement de contexte et ordonnancement



États d'un processus



```
$ ps -o pid,ppid,stat,comm
  PID   PPID  STAT COMMAND
  1200   1100  R+   bash
  1300   1200  Z    lifecycle <defunct>
```

événements

- bash (1200) fait un fork → naissance de 1300
- l'E/S de 1200 est terminée (interruption) → prêt
- 1300 se termine avec le statut 7 : il reste zombie jusqu'à ce que son parent appelle wait

Comment crée-t-on un processus?

Windows

`CreateProcess()` crée le processus et charge le programme en une seule étape, avec de nombreuses options.

Unix

1. `fork()` : un clone du processus courant
2. Le clone ajuste ce qu'il veut (descripteurs, répertoire...)
3. `exec()` : il devient un autre programme

Quatre familles: **créer** (`fork`), **exécuter** (`exec...`), **terminer et signaler** (`exit`, `kill` → U6), **attendre** (`wait...`).

Aujourd'hui aussi: `posix_spawn()` (fork + exec en un appel), `vfork()` et `clone()` / `clone3()`, le vrai appel derrière `fork` et les threads (A4.7).

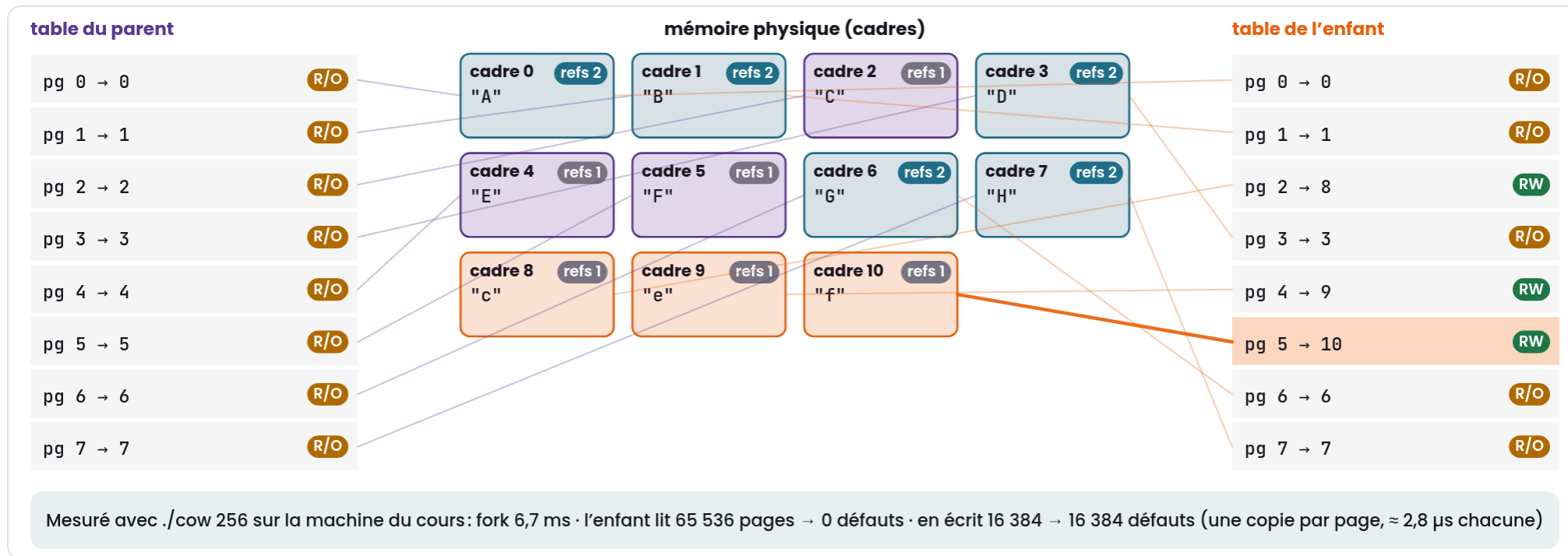
Cloner un processus: **fork** et son coût

- L'enfant exécute le même programme avec une **copie** de l'espace d'adressage du parent
- Créer le PCB coûte peu; copier chaque page coûterait très cher
- **Copie sur écriture** : elles sont partagées et copiées seulement à l'écriture

Sortie réelle

```
$ ./cow 256
parent: 256 MiB filled (65536 pages of 4096
bytes)
fork took 6.70 ms (only the page tables are
copied)
child reads 65536 pages: 0 page faults
(shared, nothing copied; sum 7864320)
child writes 16384 pages: 16384 page faults
(one copy per page), 2753 ns per copy
parent: the child exited with status 0; my
copy still says 'x'
```

La copie sur écriture, page par page



pid_t fork(void)

- Renvoie **0 dans l'enfant**, le **PID de l'enfant dans le parent** et -1 en cas d'échec
- Identiques sauf le PID et le PPID (`getpid`, `getppid`)
- L'enfant hérite des gestionnaires et du masque de signaux, **pas** des signaux en attente
- Il hérite aussi... du tampon stdio en attente

C tds/td3/fork_pid.c

```
pid_t pid = fork();
exit_if(pid == -1, "fork");

if (pid == 0) {
    printf("child %d, parent %d\n", getpid(), getppid());
    exit(EXIT_SUCCESS);
}

/* Parent: prints nothing after fork, but must wait for its child. */
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
```

Sortie réelle

```
$ ./fork_pid_0 | cat
My pid is 356446 / child 356448, parent 356446
My pid is 356446 /
```

Tampons stdio et fork

```
printf("My pid is %d / ", getpid());  
pid = fork();  
/* child */ printf("child %d, parent %d\n", ...); exit(0);  
/* parent */ waitpid(pid, ...); return 0;
```

le processus parent sort avec `exit()` : il vide son tampon

parent · stdout: tampon complet terminé

(vide)

enfant · stdout: tampon complet terminé

(vide)

stderr: sans tampon

appels système

```
[hijo] write(1, "My pid is 356446 / child 356448, parent 356446\n", 47)  
write(1, "My pid is 356446 / ", 19)
```

tube → cat → terminal

```
My pid is 356446 / child 356448, parent 356446  
My pid is 356446 /
```

sortie réelle (run.sh):

```
My pid is 356446 / child 356448, parent 356446  
My pid is 356446 /
```

Exécuter un autre programme: `execve`

C

```
int execve(const char *path, char *const argv[], char *const envp[]);  
/* execl, execlp, execl, execv, execvp, execvpe:  
   l = list, v = vector, p = search PATH, e = environment */
```

- `argv` : les arguments de `main` ; `envp` :
l'environnement (`PATH`, `HOME`, `LANG`,
`LD_LIBRARY_PATH` ...; `getenv` / `setenv`)

- S'il réussit, il **ne revient pas** : il remplace code, données, tas et pile
- Il conserve le PID, le PPID et les descripteurs ouverts (sauf ceux en `O_CLOEXEC`)
- Avec le bit SUID, l'UID effectif devient celui du propriétaire du fichier (`passwd`, `sudo`)
- S'il échoue, il renvoie -1

`fork` + `dup2` + `exec` + `wait` : le shell au travail

launch · PID 356524

en marche

ls -l demo · PID 356526

recueilli par wait

descripteurs de l'enfant

```
0 → terminal
1 → out.txt
2 → terminal
3 → -
```

`strace -f` (trace réelle, résumée)

```
356524 execve("./launch", [ "./launch", "out.txt", "ls", "-l", "demo"], ...) = 0
356524 openat(AT_FDCWD, "out.txt", O_WRONLY|O_CREAT|O_TRUNC, 0644) = 3
356524 write(3, "ls", ...) ...
356524 clone(..., flags=...|SIGCHLD) = 356526
356524 close(3) = 0
356524 wait4(356526, <unfinished ...>
356526 dup2(3, 1) = 1
356526 close(3) = 0
356526 execve("/usr/bin/ls", ...) = 0
356526 ... 19 appels de ls (openat, getdents64, close...) ...
356526 write(1, "a.txt\nb.txt\n", 12) = 12
356526 exit_group(0) = ?
356524 <... wait4 resumed>, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 356526
356524 --- SIGCHLD {si_signo=SIGCHLD, ...} ---
356524 write(2, "launch: ls exited with status 0\n", ...)
```

mot de statut de wait

0x0000

```
WIFEXITED(s) = true
WEXITSTATUS(s) = 0
WIFSIGNALED(s) = false
```

Si chaque processus naît d'un `fork`, qui est le premier?

Processus 0

Un par processeur (*idle*), construit par le noyau, en mode noyau: il initialise la mémoire virtuelle, crée les threads du noyau (`kthreadd` , PID 2) et le processus 1.

Processus 1

Il passe en mode utilisateur et exécute `/sbin/init`, qui est aujourd'hui **systemd**: il démarre les services et adopte les orphelins.

Sortie réelle

```
$ uname -srm; grep PRETTY_NAME /etc/os-release; ps -p 1 -o comm=; lsmod | tail -n +2 | wc -l; nproc; systemd-analyze
Linux 7.0.0-34-generic x86_64
PRETTY_NAME="Ubuntu 26.04 LTS"
systemd
...
```

Terminer un processus: `_exit` et `exit`

C

```
void _exit(int status);    /* system call */  
void exit(int status);    /* atexit + flush stdio + _exit */
```

Le statut (0–255) est récupéré par le parent avec `wait` ; le shell l’affiche dans `$?`. Non nul = erreur. Aucun des deux ne revient.

- Les fichiers sont fermés et les pages libérées (sauf les partagées)
- Les enfants vivants passent à `init` ou au *subreaper* le plus proche
- Le parent reçoit `SIGCHLD` (ignoré par défaut)
- S’il était chef de session avec un terminal, les autres reçoivent `SIGHUP`
- Préférez `exit` ; utilisez `_exit` dans l’enfant après un `exec` raté

Sessions et services (démons)

- Une **session** regroupe des processus (typiquement, ceux d'un terminal) avec un chef
- `setsid()` : crée une nouvelle session sans terminal de contrôle; -1 s'il est déjà chef
- Dans la session, des **groupes de processus** (une tâche du shell: `a | b`):
`setpgid / getpgid` ; `getsid` et `tcgetsid` interrogent la session d'un processus et d'un terminal
- Démon classique: `fork`, le parent se termine, l'enfant appelle `setsid`

Mise à jour 2026

Sous Linux, les services sont lancés par **systemd** : le programme tourne au premier plan et systemd gère la session, les journaux (`journalctl`), les redémarrages et les limites (cgroups). Il n'est plus nécessaire de « se démoniser ».

Attendre un enfant: `wait` et `waitpid`

C

```
pid_t wait(int *status);           /* = waitpid(-1, status, 0) */
pid_t waitpid(pid_t pid, int *status, int options);
/* pid > 0: that child · -1: any child · 0 / < -1: a process group
   options: WNOHANG (do not block) · WUNTRACED (stopped by SIGSTOP,
   SIGTSTP, SIGTTIN, SIGTTOU) · WCONTINUED (resumed by SIGCONT) */
```

Un enfant terminé reste un **zombie** jusqu'au `wait` :

Sortie réelle

```
$ ./lifecycle
1) the child 356667 has exited, but nobody has
called wait yet:
      PID    PPID STAT COMMAND
      356667 356666 Z    lifecycle
waitpid → WIFEXITED = 1, WEXITSTATUS = 7;
now ps finds nothing:
      PID    PPID STAT COMMAND
...
```

Comment s'est-il terminé? Les macros W...

WIFEXITED → terminé avec `exit`

WEXITSTATUS

WIFSIGNALED → tué par un signal

WTERMSIG

WIFSTOPPED → arrêté (avec

WSTOPSIG

WUNTRACED)

WIFCONTINUED repris (avec

WCONTINUED)

C tds/td3/launch.c

```
int status;
exit_if(waitpid(pid, &status, 0) == -1, "waitpid");
if (WIFEXITED(status)) {
    fprintf(stderr, "launch: %s exited with status %d\n",
            return WEXITSTATUS(status);
}
if (WIFSIGNALED(status)) {
    fprintf(stderr, "launch: %s killed by signal %d (%s",
            signame(WTERMSIG(status)), WCOREDUMP(status),
    return 128 + WTERMSIG(status);
}
/* sh
```

TD3 corrigé + défi IA

- Les 5 variantes de `buffer` sur un terminal et sur un tube
- `fork_pid` et ses trois corrections; `launch` avec trois fins
- Nouveau: `zones`, `lazy`, `cow` et `lifecycle` (mesures)

Bash

```
make -C tds && tds/tests/run.sh td3
```



Universidad
Indoamérica

À RETENIR

Trois questions

1. Pourquoi un fork d'un processus de 256 Mio prend-il des millisecondes ?
2. Que conserve un processus après `execve` ?
3. Qu'est-ce qu'un zombie et qui le supprime ?